



Design Patterns and Unit Testing

Names:	Garcia, Dan Matthew P. Orteza, Clayborne Benedict	Date:	September 13, 2026
Program:	BS CPE		

A. Learning Outcomes:

At the end of this laboratory exercise, students should be able to:

- Apply software design patterns in application development.
- Implement the Singleton and Factory Design Patterns.
- Create unit tests using a testing framework.
- Validate software functionality through automated testing.
- Collaborate effectively in a team-based software development project.

B. Scenario:

Paws and Care Veterinary Clinic currently manages pet appointments manually, causing scheduling conflicts and difficulty tracking records. To improve efficiency, the clinic wants a Vet Clinic Appointment Management System that allows staff to register pet owners, manage pet records, schedule appointments, and cancel bookings. The system will utilize the Singleton Pattern for managing a single clinic database, the Factory Pattern for creating different pet types, and Unit Testing to ensure all functionalities work correctly.

C. Functional Requirements

1. Pet Owner Management
 - Register new pet owners.
 - View registered pet owners.
 - Store owner information (Owner ID, Name, Contact Number).
 2. Pet Management
 - Add pet records.
 - View pet information.
 - Associate pets with their respective owners.
 - Support different pet types:
 - Dog
 - Cat
 - Bird
 - Rabbit
 3. Appointment Management
 - Schedule appointments.
 - View appointment schedules.
 - Cancel appointments.
 - Update appointment status (Scheduled, Completed, Cancelled).
 4. Design Pattern Implementation
 - Singleton Pattern
 - Create a single ClinicDatabase instance to manage all records.
 - Factory Pattern
 - Create pet objects through a PetFactory class.
 5. Unit Testing
- Develop unit tests for:
- Register Pet Owner
 - Add Pet Record
 - Schedule Appointment
 - Cancel Appointment
 - Validate Singleton Instance

Laboratory Report 4

Design Patterns and Unit Testing

(Please include the following: 1. Screenshots of the code 2. Screenshots of the output)

Duplicate if necessary

Screenshots of Code (Name of Members)

```
main.py > ...
1  # Pet owner management
2  class Owner:
3
4      def __init__(self, owner_id, name, contact_number):
5          self.owner_id = owner_id
6          self.name = name
7          self.contact_number = contact_number
8
9      def __str__(self):
10         return f"[{self.owner_id}] {self.name} - {self.contact_number}"
11
12
13  class Pet:
14
15      def __init__(self, name, owner_id):
16          self.name = name
17          self.owner_id = owner_id
18          self.species = "Unknown"
19
20      def __str__(self):
21         return f"{self.name} ({self.species}) - Owner ID: {self.owner_id}"
22
23
24  # pet management
25  class Dog(Pet):
26      def __init__(self, name, owner_id):
27          super().__init__(name, owner_id)
28          self.species = "Dog"
29
30
31  class Cat(Pet):
32      def __init__(self, name, owner_id):
33          super().__init__(name, owner_id)
34          self.species = "Cat"
35
36
```

Design Patterns and Unit Testing

```
37 class Bird(Pet):
38     def __init__(self, name, owner_id):
39         super().__init__(name, owner_id)
40         self.species = "Bird"
41
42
43 class Rabbit(Pet):
44     def __init__(self, name, owner_id):
45         super().__init__(name, owner_id)
46         self.species = "Rabbit"
47
48
49 class PetFactory:
50
51     _pet_types = {
52         "dog": Dog,
53         "cat": Cat,
54         "bird": Bird,
55         "rabbit": Rabbit,
56     }
57
58     @staticmethod
59     def create_pet(pet_type, name, owner_id):
60         pet_class = PetFactory._pet_types.get(pet_type.lower())
61         if pet_class is None:
62             raise ValueError(f"Unsupported pet type: {pet_type}")
63         return pet_class(name, owner_id)
64
65
66 # appointment management
67 class Appointment:
68
69     def __init__(self, appointment_id, pet_name, owner_id, date_time, status="Scheduled"):
70         self.appointment_id = appointment_id
71         self.pet_name = pet_name
72         self.owner_id = owner_id
73         self.date_time = date_time
74         self.status = status
75
76     def __str__(self):
77         return f"[{self.appointment_id}] {self.pet_name} (Owner: {self.owner_id}) - {self.date_time} - {self.status}"
78
79
80 # ---- input validation helpers ----
81 def is_valid_name(name):
82     """True if name has letters/spaces/hyphens/apostrophes only - no digits."""
83     stripped = name.replace(" ", "").replace("-", "").replace("'", "")
84     return stripped.isalpha() if stripped else False
85
86
87 def is_valid_contact(contact):
88     """True if contact is digits only, allowing spaces, dashes, or a leading +."""
89     stripped = contact.replace(" ", "").replace("-", "")
90     if stripped.startswith("+"):
91         stripped = stripped[1:]
92     return stripped.isdigit() and len(stripped) >= 7
93
94
95 # Singleton Pattern: one ClinicDatabase instance holds ALL clinic data
96 class ClinicDatabase:
97
98     _instance = None
99
100     def __new__(cls):
101         if cls._instance is None:
102             cls._instance = super().__new__(cls)
103             cls._instance.initialize()
104         return cls._instance
105
106     def initialize(self):
```

Design Patterns and Unit Testing

```

106     def _initialize(self):
107         self.owners = {}
108         self.pets = []
109         self.appointments = []
110         self._next_owner_id = 1
111         self._next_appointment_id = 1
112
113     # ---- owners ----
114     def register_owner(self, name, contact_number):
115         if not is_valid_name(name):
116             raise ValueError("Owner name must contain letters only (no numbers).")
117         if not is_valid_contact(contact_number):
118             raise ValueError("Contact number must contain digits only (no letters), at least 7 digits.")
119         owner_id = f"O{self._next_owner_id:03d}"
120         self._next_owner_id += 1
121         owner = Owner(owner_id, name, contact_number)
122         self.owners[owner_id] = owner
123         return owner
124
125     # ---- pets ----
126     def add_pet(self, pet_type, name, owner_id):
127         if not is_valid_name(name):
128             raise ValueError("Pet name must contain letters only (no numbers).")
129         if owner_id not in self.owners:
130             raise ValueError(f"Owner ID {owner_id} does not exist.")
131         pet = PetFactory.create_pet(pet_type, name, owner_id)
132         self.pets.append(pet)
133         return pet
134
135     def get_pet(self, owner_id, pet_name):
136         for pet in self.pets:
137             if pet.owner_id == owner_id and pet.name.lower() == pet_name.lower():
138                 return pet
139         return None
140

```

Design Patterns and Unit Testing

```

141 # ---- appointments ----
142 def schedule_appointment(self, pet_name, owner_id, date_time):
143     if owner_id not in self.owners:
144         raise ValueError(f"Owner ID {owner_id} does not exist.")
145     if self.get_pet(owner_id, pet_name) is None:
146         raise ValueError(f"No pet named '{pet_name}' found for owner {owner_id}.")
147     appointment_id = f"A{self._next_appointment_id:03d}"
148     self._next_appointment_id += 1
149     appointment = Appointment(appointment_id, pet_name, owner_id, date_time)
150     self.appointments.append(appointment)
151     return appointment
152
153 def cancel_appointment(self, appointment_id):
154     for appt in self.appointments:
155         if appt.appointment_id == appointment_id:
156             appt.status = "Cancelled"
157             return appt
158     raise ValueError(f"Appointment ID {appointment_id} not found.")
159
160 def update_appointment_status(self, appointment_id, new_status):
161     valid_statuses = ("Scheduled", "Completed", "Cancelled")
162     if new_status not in valid_statuses:
163         raise ValueError(f"Status must be one of {valid_statuses}")
164     for appt in self.appointments:
165         if appt.appointment_id == appointment_id:
166             appt.status = new_status
167             return appt
168     raise ValueError(f"Appointment ID {appointment_id} not found.")
169
170
171 def print_menu():
172     print("\n==== Paws and Care Veterinary Clinic =====")
173     print("1. Register Pet Owner")
174     print("2. View Pet Owners")
175     print("3. Add Pet Record")

```

Design Patterns and Unit Testing

```
176     print("4. View Pets")
177     print("5. Schedule Appointment")
178     print("6. View Appointments")
179     print("7. Cancel Appointment")
180     print("8. Update Appointment Status")
181     print("0. Exit")
182
183
184     def run_app():
185         db = ClinicDatabase()
186
187         while True:
188             print_menu()
189             choice = input("Enter choice: ").strip()
190
191             if choice == "1":
192                 name = input("Owner name: ").strip()
193                 contact = input("Contact number: ").strip()
194                 try:
195                     owner = db.register_owner(name, contact)
196                     print(f"Registered: {owner}")
197                 except ValueError as e:
198                     print(f"Error: {e}")
199
200             elif choice == "2":
201                 if not db.owners:
202                     print("No owners registered yet.")
203                 for o in db.owners.values():
204                     print(o)
205
206             elif choice == "3":
207                 owner_id = input("Owner ID: ").strip()
208                 pet_type = input("Pet type (Dog/Cat/Bird/Rabbit): ").strip()
209                 name = input("Pet name: ").strip()
210                 try:
```



Design Patterns and Unit Testing

```
211         pet = db.add_pet(pet_type, name, owner_id)
212         print(f"Added pet: {pet}")
213     except ValueError as e:
214         print(f"Error: {e}")
215
216     elif choice == "4":
217         owner_id = input("Filter by Owner ID (blank = all): ").strip() or None
218         filtered = [p for p in db.pets if owner_id is None or p.owner_id == owner_id]
219         if not filtered:
220             print("No pets found.")
221         for p in filtered:
222             print(p)
223
224     elif choice == "5":
225         owner_id = input("Owner ID: ").strip()
226         pet_name = input("Pet name: ").strip()
227         date_time = input("Date and time (e.g., 2026-09-20 14:30): ").strip()
228         try:
229             appt = db.schedule_appointment(pet_name, owner_id, date_time)
230             print(f"Scheduled: {appt}")
231         except ValueError as e:
232             print(f"Error: {e}")
233
234     elif choice == "6":
235         if not db.appointments:
236             print("No appointments yet.")
237         for a in db.appointments:
238             print(a)
239
240     elif choice == "7":
241         appointment_id = input("Appointment ID to cancel: ").strip()
242         try:
243             appt = db.cancel_appointment(appointment_id)
244             print(f"Cancelled: {appt}")
245         except ValueError as e:
```



Design Patterns and Unit Testing

```
232         print(f"Error: {e}")
233
234     elif choice == "6":
235         if not db.appointments:
236             print("No appointments yet.")
237         for a in db.appointments:
238             print(a)
239
240     elif choice == "7":
241         appointment_id = input("Appointment ID to cancel: ").strip()
242         try:
243             appt = db.cancel_appointment(appointment_id)
244             print(f"Cancelled: {appt}")
245         except ValueError as e:
246             print(f"Error: {e}")
247
248     elif choice == "8":
249         appointment_id = input("Appointment ID: ").strip()
250         new_status = input("New status (Scheduled/Completed/Cancelled): ").strip()
251         try:
252             appt = db.update_appointment_status(appointment_id, new_status)
253             print(f"Updated: {appt}")
254         except ValueError as e:
255             print(f"Error: {e}")
256
257     elif choice == "0":
258         print("Exiting. Goodbye!")
259         break
260
261     else:
262         print("Invalid choice. Please try again.")
263
264
265 if __name__ == "__main__":
266     run_app()
```

*Resize the image if necessary.

Screenshots of Output



Design Patterns and Unit Testing

<pre>===== Paws and Care Veterinary Clinic ===== 1. Register Pet Owner 2. View Pet Owners 3. Add Pet Record 4. View Pets 5. Schedule Appointment 6. View Appointments 7. Cancel Appointment 8. Update Appointment Status 0. Exit Enter choice: 1 Owner name: neil Contact number: 09123456789 Registered: [0001] neil - 09123456789 ===== Paws and Care Veterinary Clinic ===== 1. Register Pet Owner 2. View Pet Owners 3. Add Pet Record 4. View Pets 5. Schedule Appointment 6. View Appointments 7. Cancel Appointment 8. Update Appointment Status 0. Exit Enter choice: 2 [0001] neil - 09123456789 ===== Paws and Care Veterinary Clinic ===== 1. Register Pet Owner 2. View Pet Owners 3. Add Pet Record 4. View Pets 5. Schedule Appointment 6. View Appointments 7. Cancel Appointment</pre>	
--	--



Design Patterns and Unit Testing

```
8. Update Appointment Status
0. Exit
Enter choice: 3
Owner ID: 0001
Pet type (Dog/Cat/Bird/Rabbit): Dog
Pet name: Buldak
Added pet: Buldak (Dog) - Owner ID: 0001

===== Paws and Care Veterinary Clinic =====
1. Register Pet Owner
2. View Pet Owners
3. Add Pet Record
4. View Pets
5. Schedule Appointment
6. View Appointments
7. Cancel Appointment
8. Update Appointment Status
0. Exit
Enter choice: 4
Filter by Owner ID (blank = all):
Buldak (Dog) - Owner ID: 0001

===== Paws and Care Veterinary Clinic =====
1. Register Pet Owner
2. View Pet Owners
3. Add Pet Record
4. View Pets
5. Schedule Appointment
6. View Appointments
7. Cancel Appointment
8. Update Appointment Status
0. Exit
Enter choice: 5
Owner ID: 0001
Pet name: Buldak
Date and time (e.g., 2026-09-20 14:30): 2026-09-20 15:50
Scheduled: [A001] Buldak (Owner 0001) - 2026-09-20 15:50 - Scheduled
```



Design Patterns and Unit Testing

```
===== Paws and Care Veterinary Clinic =====
1. Register Pet Owner
2. View Pet Owners
3. Add Pet Record
4. View Pets
5. Schedule Appointment
6. View Appointments
7. Cancel Appointment
8. Update Appointment Status
0. Exit
Enter choice: 6
[A001] Buldak (Owner 0001) - 2026-09-20 15:50 - Scheduled

===== Paws and Care Veterinary Clinic =====
1. Register Pet Owner
2. View Pet Owners
3. Add Pet Record
4. View Pets
5. Schedule Appointment
6. View Appointments
7. Cancel Appointment
8. Update Appointment Status
0. Exit
Enter choice: 7
Appointment ID to cancel: A001
Cancelled: [A001] Buldak (Owner 0001) - 2026-09-20 15:50 - Cancelled

===== Paws and Care Veterinary Clinic =====
1. Register Pet Owner
2. View Pet Owners
3. Add Pet Record
4. View Pets
5. Schedule Appointment
6. View Appointments
7. Cancel Appointment
8. Update Appointment Status
0. Exit
Enter choice: 5
Owner ID: 0001
```



Design Patterns and Unit Testing

<pre>Pet name: Buldak Date and time (e.g., 2026-09-20 14:30): 2026-09-19 15:00 Scheduled: [A002] Buldak (Owner 0001) - 2026-09-19 15:00 - Scheduled ===== Paws and Care Veterinary Clinic ===== 1. Register Pet Owner 2. View Pet Owners 3. Add Pet Record 4. View Pets 5. Schedule Appointment 6. View Appointments 7. Cancel Appointment 8. Update Appointment Status 0. Exit Enter choice: 7 Appointment ID to cancel: Error: Appointment ID not found. ===== Paws and Care Veterinary Clinic ===== 1. Register Pet Owner 2. View Pet Owners 3. Add Pet Record 4. View Pets 5. Schedule Appointment 6. View Appointments 7. Cancel Appointment 8. Update Appointment Status 0. Exit Enter choice: 8 Appointment ID: A001 New status (Scheduled/Completed/Cancelled): Completed Updated: [A001] Buldak (Owner 0001) - 2026-09-20 15:50 - Completed ===== Paws and Care Veterinary Clinic ===== 1. Register Pet Owner 2. View Pet Owners 3. Add Pet Record 4. View Pets 5. Schedule Appointment</pre>	
---	--



Design Patterns and Unit Testing

```
6. View Appointments
7. Cancel Appointment
8. Update Appointment Status
0. Exit
Enter choice: 6
[A001] Buldak (Owner 0001) - 2026-09-20 15:50 - Completed
[A002] Buldak (Owner 0001) - 2026-09-19 15:00 - Scheduled

===== Paws and Care Veterinary Clinic =====
1. Register Pet Owner
2. View Pet Owners
3. Add Pet Record
4. View Pets
5. Schedule Appointment
6. View Appointments
7. Cancel Appointment
8. Update Appointment Status
0. Exit
Enter choice: █
```

*Resize the image if necessary.



Design Patterns and Unit Testing

D. Findings, Observations, and Comments:

Guide Questions:

1. What are your Group's realizations in this laboratory?
2. What are the advantages of using design patterns in software development?
3. What challenges did you face during testing phases?

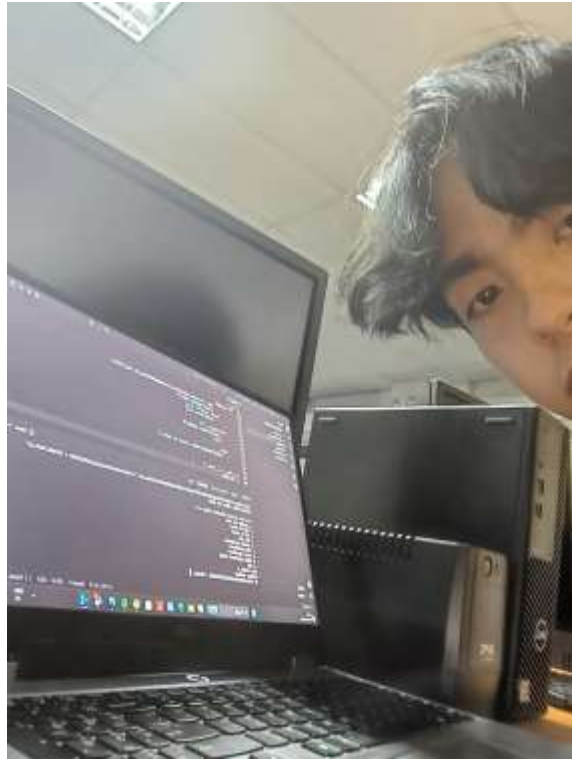
Working through this exercise made the theoretical side of design patterns click in a way that reading about them alone doesn't. Singleton and Factory are often described abstractly in lectures, but implementing ClinicDatabase as an actual single point of truth for owners, pets, and appointments, and then verifying with `assertIs()` that two separate calls to `ClinicDatabase()` return the same object has made the "why" concrete instead of just memorized. We also realized that a working demo isn't the same as a correct one: our first version ran without errors but silently accepted malformed input, like letters in a contact number or digits in a name, which only surfaced once we deliberately tried to break it rather than just running the happy path.

That realization ties directly into why design patterns matter in the first place. The clearest advantage is that patterns give a proven, communicable solution to a recurring problem instead of everyone improvising their own structure. Singleton guaranteed that clinic data couldn't accidentally fragment into two disconnected stores, a real risk if multiple parts of the program tried to manage owners and appointments independently. Factory, meanwhile, decoupled the rest of the program from the details of how a Dog differs from a Cat internally; anything that needs a pet just asks the factory for one, so adding a new species later means touching one place instead of hunting through the whole codebase. Beyond the immediate code, patterns also made collaboration easier, since naming something "the Singleton" or "the Factory" communicates its role instantly to anyone else who's studied the same vocabulary.

That same property, though, became the main source of friction once we moved into testing. Because ClinicDatabase persists across calls by design, state kept leaking between tests, a test asserting "there should be exactly one owner" could fail simply because an earlier test had already registered one against the same underlying instance. That forced us to add an explicit `reset()` method just to give each test a clean slate, which was a useful lesson in itself: the exact behavior that makes Singleton valuable in production, persistence across calls, is what makes it awkward to isolate in a test suite. A second, more mundane challenge was environment setup rather than logic, resolving import errors caused by the workspace root not matching the folder the files actually lived in, a good reminder that "the code is correct" and "the code runs" aren't guaranteed to be the same thing until the environment is configured properly.

Laboratory Report 4
Design Patterns and Unit Testing

PICTURE/PROOF OF DOING THE ACTIVITY





Laboratory Report 4

Design Patterns and Unit Testing

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indention, but inappropriate elements and structures and poor program documentation.	The program had corrected indention, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indention, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings. observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings. observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings. observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings. observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED			
Rating		Signature	
		Date	